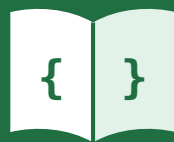


---

# Module 14 — Testing Spring Boot Applications

Exercises

Spring Boot Edu



2026-09-24

## Contents

|                                                     |          |
|-----------------------------------------------------|----------|
| <b>How to Work</b>                                  | <b>2</b> |
| <b>Exercise 1 — Test an Untested Service (Easy)</b> | <b>2</b> |
| <b>Exercise 2 — Fix Flaky Tests (Medium)</b>        | <b>3</b> |
| <b>Exercise 3 — Write Architecture Rules (Hard)</b> | <b>3</b> |
| <b>Extra Challenge (Optional)</b>                   | <b>4</b> |

## How to Work

In this module, **you write the tests**. The code under test is given and must not be changed.

1. The starter code is in `modules/14-testing/exercise/`. The `TODO` comments are in `src/test/`.
2. The exercises need no Docker.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/14-testing/exercise -am test
```

4. The exercise is done when all tests are green. That includes the checks that come with the exercises (`Exercise1MutantsTest`, `Exercise3RulesCatchLegacyTest`), which make sure your tests would actually catch mistakes.
5. If you get stuck, read the hints first, then the solution in `modules/14-testing/solution/src/test/`.

## Exercise 1 — Test an Untested Service (Easy)

**Goal:** `LoyaltyCalculator` calculates loyalty points. It has no tests yet. Its Javadoc describes five rules. Write tests that would catch a mistake in any of them.

**Tasks** (`exercise1.LoyaltyCalculatorTest`, in `src/test`):

- `TODO 1a` — the minimum order (19.99 and exactly 20.00) and “only full 10.00 count” (29.99).
- `TODO 1b` — the same order for BRONZE, SILVER and GOLD. Mind the rounding for SILVER.
- `TODO 1c` — the birthday bonus.
- `TODO 1d` — the upper limit of 500 points.

**Hints:**

- Lesson section 3.1: `@ParameterizedTest` with `@CsvSource` tests many cases in few lines.
- Always create the calculator with `Subjects.loyaltyCalculator()`. `Exercise1MutantsTest` swaps it for three **deliberately broken** calculators and expects your tests to fail against each of them. If a mutant survives, a rule is not tested well enough.
- Boundaries find the most bugs: test exactly at the limit, and one step below it.

**Acceptance criteria:** `LoyaltyCalculatorTest` passes, and `Exercise1MutantsTest` passes (all three mutants are caught).

**Estimated time:** 30 minutes

## Exercise 2 — Fix Flaky Tests (Medium)

**Goal:** `DeliveryEstimatorTest` contains three tests that pass on some days, runs or machines and fail on others. Make each of them deterministic **without changing the production code**.

**Tasks** (`exercise2.DeliveryEstimatorTest`):

- TODO 2a — the test depends on today's date. It fails when a weekend falls into the next three days.
- TODO 2b — the test depends on the iteration order of a `HashSet`, which is not guaranteed.
- TODO 2c — the test waits a fixed 200 ms, but the dispatch takes 50-400 ms.

**Hints:**

- `DeliveryEstimator` accepts a `Clock`: `Clock.fixed(Instant.parse("2026-09-21T10:00:00Z"), ZoneOffset.UTC)` is always the same Monday. Then write one test for a weekday and one across a weekend.
- `AssertJ: containsExactlyInAnyOrder(...)`.
- `Awaitility: await().atMost(Duration.ofSeconds(2)).until(() -> ...)` waits only as long as needed.
- A good check: run the test class ten times in a row (IntelliJ: Run until failure).

**Acceptance criteria:** all 3 tests in `DeliveryEstimatorTest` pass, every time.

**Estimated time:** 30 minutes

## Exercise 3 — Write Architecture Rules (Hard)

**Goal:** the package `exercise3.shop` is built in layers: `web` → `service` → `repository`. The package `exercise3.legacy` breaks these rules. Write `ArchUnit` rules that pass for the shop and would catch the legacy code.

**Tasks** (`exercise3.ShopArchitectureTest`):

- TODO 3a — classes in a repository package may only be used by the service (and repository) packages.
- TODO 3b — classes whose name ends with `Controller` must not depend on the repository package.
- TODO 3c — classes named `...Repository` or `...Store` must live in a repository package.

**Hints:**

- Lesson section 3.8 shows the style: `noClasses().that()...should()...` and `classes().that()...should()`
- 3a: `...should().onlyBeAccessed().byAnyPackage("..service..", "..repository..")`.

- 3c: `.that().haveSimpleNameEndingWith("Repository").or().haveSimpleNameEndingWith("Store")`.
- `ShopArchitectureTest` checks only the shop package. `Exercise3RulesCatchLegacyTest` applies your rules to the whole exercise, including legacy, and expects every rule to find a violation.

**Acceptance criteria:** `ShopArchitectureTest` and `Exercise3RulesCatchLegacyTest` pass.

**Estimated time:** 40 minutes

## Extra Challenge (Optional)

Look at the three mutants in `solution/src/test/.../exercisel/mutants`. Invent a fourth bug in `LoyaltyCalculator` that your tests would **not** catch, add it as a mutant, and then add the test that catches it. Tools such as PIT ([pitest.org](https://pitest.org)) do this automatically for the whole code base.